Loom: A Distributed Expert-Cache Architecture for Frontier Mixture-of-Experts Models

Technical design and preliminary validation

Author: Parthasarathy Ramanujam **Draft v0.2, 2026-07-20 Status:** design complete for v1; partial implementation; preliminary single-machine and localhost validation. Implementation progress is tracked in ROADMAP. The wire protocol is specified in SPEC. A dated design-decision log for contributors is in Appendix A.

Contents

Abstract

The strongest open-weight language models are now Mixture-of-Experts (MoE) architectures: Kimi K2 [3], DeepSeek V3 [2], and the GLM family [4], with hundreds of billions to trillions of parameters. At usable quantization their routed experts alone run to hundreds of gigabytes. They do not fit on any commodity machine, and running them today means renting datacenter GPUs that price out individuals and small teams. Multi-machine alternatives exist, but the prevailing approach splits the computation across nodes and ships activations between them every layer of every token. That places the network on the serial critical path, so a slow or lost peer stalls the whole pipeline. The problem Loom addresses is how to serve a frontier MoE model that no single affordable machine can hold, across a pool of ordinary machines on ordinary networks, without putting the network in the critical path of every token.

The opportunity is structural. An MoE model activates only a few percent of its weights per token, and those weights form an immutable, content-addressable corpus: a sparsely accessed blob store rather than a monolithic tensor. Loom keeps computation node-local and moves only weights. The corpus is sharded into content-addressed expert blocks, replicated across coordinated committees of nodes, verified by Merkle digests at every hop, and paged into the token loop on demand. Because only immutable weights cross the network, a lost peer costs a refetch from a replica rather than a broken pipeline.

What has been demonstrated. The inference engines produce correct output on reference checkpoints: the tinyllamas models, and DeepSeek-V2-Lite, a 15.7-billion-parameter MoE, at Q4_K_M quantization. A node holding 33% of that model's shards produced the expected, token-identical completion while fetching the missing experts from a single LAN peer during inference, with every block digest-verified before use. This is a correctness-and-

integration result on localhost with scalar CPU kernels (about 0.3 tokens/sec on one core), not a throughput or wide-area result (Section 10).

What is targeted, and not yet demonstrated. The design goal is serving GLM-5.2-class models (744 billion parameters, about 19,200 experts) on swarms of 16 to 32 GB machines. That model has not been run, kernels are not yet vectorized, and there is no batching. One caveat governs expectations throughout (Section 9): on ordinary gigabit Ethernet, distribution buys capacity, meaning the model fits where it otherwise could not, but not speed. Competitive throughput additionally requires 10-gigabit-class fabric, a pinned hot set, and batched serving.

Glossary

Term	Meaning
Expert	One routed feed-forward sub-network (gate/up/down matrices) in one MoE layer. GLM 5.2 has 256 per layer; 8 activate per token.
Expert shard	The unit of distribution and compute: one expert's weights, about 19 MB at int4. A fetched expert shard is exactly what a MoE matmul consumes.
Resident chunk	A roughly 16 MB piece of the resident bundle, the non-expert weights (attention, embeddings, shared experts, router, norms) that every full node holds in full.
Manifest / version id	The list of shard digests plus layout. Its Merkle root is the model's version identity, which nodes pin requests to.
Full node	Holds the resident bundle and a subset of expert shards, runs local inference, serves shards to peers. The supply side.
Light node	Holds no weights and runs no engine; exposes the same local API and delegates to full nodes. The demand side.
Committee	A group of full nodes that collectively holds the complete shard set, a self-sufficient serving cell.
MLA	Multi-head Latent Attention [1]: attention with a low-rank compressed key/value cache (about 576 values per token per layer), used by the target models.
Mesh	The peer table each node builds from gossip announcements, used for discovery and fallback routing.

---

1. Introduction

Loom targets operators who want to run frontier open-weight MoE models on hardware they already have: a homelab, a research group's workstations, or a small trusted cluster, rather than rented datacenter GPUs. It is not built for interactive single-user chat on a cold model over a slow link (Section 9 explains why), and v1 assumes a cooperative, operator-run swarm rather than an open adversarial network (Section 7).

Three facts frame the problem.

1. **The best open models are MoE.** Kimi K2 (about 1T total, 32B active), DeepSeek V3 (671B, 37B), and the GLM family activate roughly 3 to 5 percent of their parameters per token [2][3][4].
2. **The weights do not fit cheaply.** At int4, the routed experts of a GLM-5.2-class model total about 370 GB, beyond any commodity machine and beyond most workstations.
3. **The activated set is small and skewed.** A single token reads on the order of 600 expert blocks (about 11.4 GB for GLM 5.2), drawn Zipf-like from a fixed corpus of roughly 19,200. This is a sparsely accessed, immutable, content-addressable blob store.

Loom's premise follows. This is a distributed storage and caching problem in the shape of an inference problem. The corpus is stored once across a swarm, replicated by policy, cryptographically verified, and paged into

computation on demand, the way a distributed filesystem treats cold blocks rather than the way a tensor-parallel runtime treats a partitioned weight matrix.

---

2. Related work

Systems that run large models across multiple machines can be organized by their unit of distribution and the failure and trust properties that follow from it.

System	Unit distributed	What crosses the network per token	Failure model	Commodity-NIC friendly
Petals [5]	Transformer layers	Activations, every layer	Serial: a slow or lost stage stalls the token	Partially (activations are small but on the critical path)
exo [6]	Layer partitions	Activations	Serial pipeline	Partially
FlexGen [9]	Offloaded tensors (single node)	None (host/device)	N/A (not distributed)	N/A
PowerInfer [8]	Hot/cold neurons (single node)	None (GPU/CPU split)	N/A	N/A
llama.cpp [7]	None (single node) or layer-split	Activations if split	Serial if split	Yes single-node
Loom	**Expert block (weights)**	**Expert blocks only** (cacheable, immutable)	**Soft: re-fetch from a replica**	**Yes for capacity; speed needs 10 GbE**

Loom reuses the GGUF weight format and GGML quantization layouts [7] and the DeepSeek MLA and sparse-routing model shape [1][2]. It introduces three things: the expert block as the unit of distribution, committee-based redundancy with coverage by construction, and a token-loop fetch that doubles as replication. Its closest philosophical relative is content-addressed storage (Merkle-verified immutable blocks [10]) applied to model weights rather than files.

Layer-split systems such as Petals and exo are deployed and effective. The distinction Loom draws is structural, not a claim that they do not work. Because they ship activations, which are mutable, per-request, and order-dependent, the network sits on the serial critical path. Loom ships only immutable weights, so a lost peer costs a re-fetch rather than a broken pipeline.

---

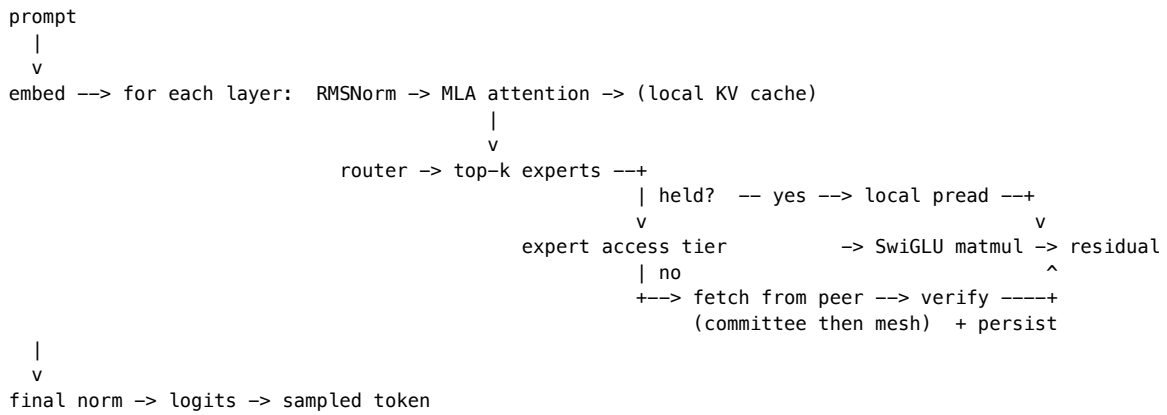
3. Core thesis: experts flow, not activations

Every full node runs the complete forward pass locally. The only data that crosses the network during inference is an expert block: immutable, content-addressed, cacheable, and identical for every user of the model. Three properties follow.

- **Soft failure.** A lost peer costs a re-fetch from a replica, not a broken pipeline. A lone node still functions, streaming experts from its own disk.
- **Compounding cache.** Frequently used experts come to reside on many nodes, so the network is touched only on the cold tail of the access distribution.
- **KV stays home.** The target models use MLA [1], which compresses the key/value cache to about 576 values per token per layer. Distributing that mutable, latency-critical, per-session state would put it on the network for a negligible capacity gain, so Loom does not.

The governing caveat, developed in Section 9: on gigabit Ethernet the swarm buys capacity, not speed. Loom is a serving-throughput architecture first and an interactive-latency one second.

Figure 1. Inference data path (one full node). Only routed-expert reads may leave the machine; everything else is local.



4. System architecture

Loom is a single binary written in Zig. It separates a data plane (weights at rest and in flight, and the local forward pass) from a control plane (membership, gossip, repair, metering). Two inference engines share a common set of quantized-matmul kernels.

- The **MLA** engine (`deepseek2`), covering the DeepSeek/Kimi family: MLA attention, YaRN context extension [11], and a byte-level BPE tokenizer read from GGUF metadata. Validated on real DeepSeek-V2-Lite weights (Q4_K_M).
- The **GQA** engine, covering `llama` (including Mixtral), `qwen2moe`, `qwen3moe` and `glm4moe` in one forward pass. These architectures differ only in optional pieces over a shared skeleton, so the engine detects them from the tensors a file contains rather than from per-architecture assumptions: QKV biases, per-head Q/K normalization, dense or mixture-of-experts FFN, an optionally sigmoid-gated shared expert, leading dense layers, a post-attention norm in place of `ffn_norm`, and trailing MTP blocks that are skipped. Only the rotary-embedding style is pinned per architecture, because an incorrect choice yields fluent but wrong output rather than an error. The dense llama path is validated against the tinyllamas reference models at F32, Q4_0 and Q8_0.

Both engines share one router (sigmoid or softmax gating, selection bias, top-k with optionally renormalized and scaled gates, shared experts), so a new architecture requires an attention variant rather than a new engine. The distribution plane was never architecture-specific: expert sharding keys on the GGUF tensor-naming convention that every mixture-of-experts conversion follows.

Weights remain in their GGML quantized formats in a read-only memory map, and each matmul is a fused, vectorized kernel over the raw bytes, distributed across a worker pool by row, so no tensor is dequantized wholesale. Two families are supported. *Affine* formats (F32/F16/Q4_0/Q5_0/Q8_0/Q4_K/Q5_K/Q6_K) reconstruct a value arithmetically. *Codebook* formats (IQ1_S, IQ1_M, IQ2_XXS, IQ2_XS, IQ2_S, IQ3_XXS, IQ3_S, IQ4_NL, IQ4_XS, and MXFP4) instead store an index into a static grid table, so a transcription error in that table would silently corrupt weights rather than fail; every codebook decoder is therefore checked bit-for-bit against the reference implementation's output on stored vectors. The source layout appears in Appendix B.

5. Expert-aligned sharding

The unit of distribution equals the unit of computation. An expert shard is one expert's gate/up/down matrices in one layer, described as a byte-extent list over the GGUF file's three-dimensional expert tensors. A fetched expert shard is precisely what a MoE matmul consumes: no reassembly, no erasure coding, no partial reads on the token path.

Everything that is not a routed expert (attention, norms, shared experts, embeddings, router weights, the file header) forms the resident bundle, split into roughly 16 MB resident chunks that every full node holds in full.

This is what keeps compute node-local. Any node can run the dense path and the router by itself; only routed-expert reads may leave the machine.

Every byte of the file belongs to exactly one shard, either an expert shard or a resident chunk. Each shard carries a SHA-256 digest, and the Merkle root over the digest list, together with the layout, is the model's version identity, which nodes gossip and pin their requests to. Integrity is therefore free at every hop: a shard from any source is checked against the local manifest before it reaches disk or a matmul.

Sizing for the GLM-5.2-class target:

Quantity	Value
Expert shards	19,200 (75 MoE layers x 256 experts)
Expert-shard size	about 19 MB (int4)
Routed corpus	about 370 GB, stored once across the swarm
Resident bundle	about 10 GB, on every full node
Logical file	about 380 GB; each node's copy is sparse (about 60 GB of real bytes at defaults)
Manifest / holdings bitmap	about 1 MB / 2.4 KB

Measured on real DeepSeek-V2-Lite Q4_K_M (10.4 GB): 1,664 expert shards of 4.98 to 6.02 MB, plus 73 resident chunks totaling 0.78 GB; manifest 204 KB; holdings bitmap 218 B.

---

6. Distribution protocol

This section states the protocol's shape and invariants. Field-level detail is in SPEC.

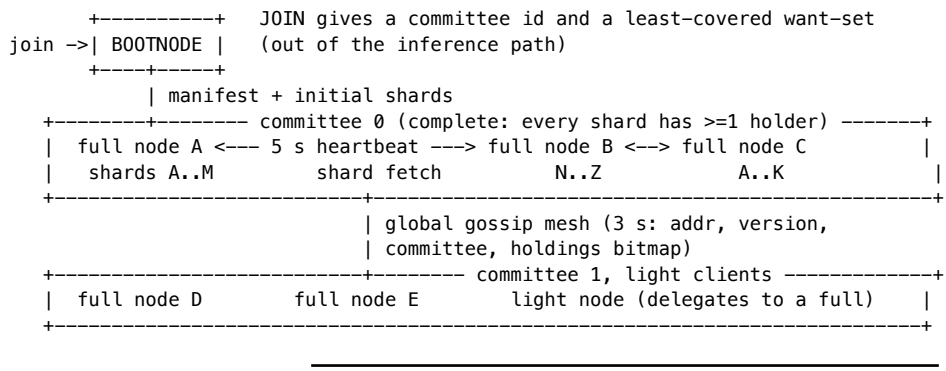
Component	Role	Key property
Bootnode	Onboards joiners; assigns each a committee and a least-covered-first want-set	Coverage by construction; not in the inference path; trusted in v1
Committee	A group of full nodes that collectively holds every shard	Completeness means at least one holder per shard; filled toward redundancy R (default 2)
Gossip mesh	Every node announces its holdings every 3 s; peers merge into a peer table	Transitive discovery; carries committee membership
Query path	Fetch a missing shard: committee members first, then the mesh	Digest-verified; fails only if no reachable holder exists anywhere
Eager repair	A 2 s loop re-fetches any wanted-but-missing shard from all known peers	Dead peers are retried, not forgotten; a returning holder is drained within one tick
Wire messages	Binary frames (Heartbeat, Announce, ExpertRequest/Response) with adaptive Snappy [13]	Requests pin the version, so a cross-version peer is refused (the hardfork guard)

Two design choices hold the protocol together.

Coverage by construction. Rather than hope random placement covers every shard, the bootnode assigns each joiner the shards its committee currently covers least. Completeness (every shard has a holder) and redundancy (R holders) are distinct thresholds a committee crosses as it fills. When all committees are saturated at R, the next joiner opens a new one.

Membership is gossip-derived, not static. Announcements carry committee ids, so earlier members learn later joiners automatically and the committee view survives the bootnode's departure. Heartbeats (5 s) carry liveness plus the manifest version, a monotonic holdings sequence number, a bitmap digest, and a load hint. That is enough to detect staleness and spread load without shipping the full bitmap on every beat.

Figure 2. Committee and mesh.



---

7. Trust and security model

v1 assumes a cooperative, operator-run swarm and provides cryptographic integrity relative to a trusted manifest.

Assumptions. The bootnode is operated by the swarm operator, peers report their holdings honestly, and the transport is a trusted LAN.

Guarantees. Every shard is checked against its manifest digest before disk or matmul, so corruption, bit-rot, or a wrong-bytes peer is caught at every hop. The manifest's version id binds the shard layout, so a manifest whose extents do not tile the file exactly is rejected on parse. Version pinning on every request isolates model versions and, later, hardforks.

Non-guarantees. Which manifest is trusted is not free. Content addressing secures integrity within a manifest, not the choice of manifest. A node adopts its Merkle root on first contact (trust-on-first-use) or from operator config, and a hostile first contact could pin an alternate, internally consistent but poisoned root. Holdings claims are also unverified: a peer can advertise shards it lacks. This is caught reactively, when a fetch returns wrong or absent bytes and the requester falls through to the next peer, so committee completeness is a construction-time property under honest participation, not a cryptographic invariant. v1 does not defend against availability attacks (service refusal, mesh poisoning with fake holders, request flooding).

v2 (designed, not built): untrusted peers. The planned untrusted-peer layer adds rateless (RLNC) wide-area propagation gated behind homomorphic-hash pollution defenses [14], redundant recompute with M-of-N voting on sampled tokens, and TEE attestation. Erasure and network coding are confined to the propagation and durability planes; they are never permitted on the token path, where only direct fetch of an original block is allowed.

Known limitations of the current implementation are consolidated in Section 11.

---

8. Node classes and compensation

Loom defines two node classes.

Full nodes hold the resident bundle and their assigned expert shards, join committees, serve shards to peers, and run inference. They are the supply side. The "compute node-local, only expert blocks cross the network" property of Section 3 applies to the full-node inference data path; request, response, and control traffic naturally also cross the network.

Light nodes hold no weights, no store, and no engine, a footprint of a few megabytes suitable for a low-memory device. A light node exposes the same local API a full node does, so applications are unaware of the distinction, and delegates each request to a full node with round-robin failover across its configured backends. It stamps its own client identity on every request, and a caller-supplied identity is dropped. Light nodes are thin clients, not swarm participants: they contribute no storage or serving, so capacity planning must size the full-node fleet for aggregate light-node demand.

Compensation. Full nodes are meant to be compensated by light nodes for the inference they serve. v1 implements the accounting mechanics and leaves settlement as an integration seam. Each full node keeps a per-client ledger, where allowance equals free quota plus purchased credits minus token usage. A metered response

carries its cost and the remaining balance, and an exhausted client is refused with `payment_required` before any compute. Credits are added through an admin-gated `credit` operation whose proof field a real payment rail would verify. This is scoped as a trusted-operator accounting demonstration, not a settlement system. Its limitations are enumerated in Section 11.

---

9. Deployment and performance model

Held shards are a disk budget served by `pread`. RAM carries only the compute working set.

Resource	Minimum	Recommended
RAM	16 GB (about 10 GB mmap'd resident bundle, 0.7 GB MLA KV, 2 to 3 GB expert cache)	24 to 32 GB
Disk	75 GB NVMe (resident bundle plus about 50 GB expert shards)	150 GB NVMe
Experts held	about 1,300 (25 GB)	about 2,600 (50 GB), 13.7% of GLM 5.2
Network	1 GbE (capacity only)	10 GbE+ (also latency)

The 16 GB minimum is conditional on the resident bundle. A per-tensor-class breakdown from the real converted GLM 5.2 file is required before it is asserted as a default rather than a target. For DeepSeek-V2-Lite the measured resident bundle is 0.78 GB.

Swarm sizing against redundancy R over the 370 GB corpus (R = 3 target, R = 2 floor): at 50 GB per node, completeness (R = 1) needs at least 8 nodes, R = 2 needs at least 15, and R = 3 needs at least 22. Eight nodes at 100 GB reach R = 2.

Performance model (analytical, not yet measured). Per-token time is about `t_compute + misses * t_fetch`, where `misses` is the number of a token's routed experts absent from the local store and page cache, and `t_fetch` is about `RTT + shard_size / bandwidth` per miss. Fetches within a layer parallelize, so the layer cost is the maximum, not the sum, over its 8 or fewer misses. A cold 19 MB miss costs roughly 160 ms on 1 GbE and 16 ms on 10 GbE before protocol overhead. The consequence is direct. With a cold working set on gigabit Ethernet a single stream is seconds per token, unusable interactively. Serving becomes viable insofar as pinning, the Zipfian access skew, and organic replication drive the steady-state miss rate toward zero, and batching amortizes what remains. The `t_compute` term is itself scalar today; hardware-tailored backends (CPU SIMD, and GPU via Metal/Vulkan/CUDA, staged behind one interface in roadmap issues #10 to #14) are the lever that reduces it. No network-tier throughput measurements exist yet; Section 10 lists them as the primary measurement agenda.

Positioning follows from the model: pool commodity machines so a frontier MoE fits where it otherwise could not; speed is a separate problem, bound by network and pinning. The differentiators against layer-split systems are structural (weights not activations, soft failure, verified weights, self-healing membership). The differentiator against single-box streaming is aggregate capacity.

---

10. Preliminary results

These are early results, a case study plus live single-machine and localhost multi-node runs, not a throughput evaluation. The evidence and its boundaries:

Claim	Configuration	Evidence	Boundary
Engine correctness	DeepSeek-V2-Lite Q4_K_M; tinyllamas F32/Q4_0/Q8_0	Coherent, expected completions on real weights (validates kernels, MLA, RoPE convention, K-quants, BPE, YaRN)	Not a factual-accuracy benchmark; single-sequence; scalar kernels

Claim	Configuration	Evidence	Boundary
Expert-aligned sharding	Real 10.4 GB model	Round-trips to 1,664 expert shards plus 73 resident chunks; layout partition property-tested	Fully covered
Partial-store inference	33% store (573 of 1,737 shards), localhost, one peer, scalar	Token-identical to a full-copy run while fetching 641 experts (3.5 GB, about 29 ms/fetch, 0 failures); grew to 1,214 shards held	Not NIC throughput or failover under WAN-like RTT and loss
Committee and repair behavior	Live multi-node runs	Observed: bootstrap, gossip discovery, committee formation and saturation, heartbeat death detection, eager repair, mesh fallback	Not adversarial availability
Weight-integrity defenses	Live	A corrupted on-disk shard is caught on store-open, cleared, and re-fetched; malicious non-partition manifests rejected on parse	Fully covered
Metering	Live, two full nodes and one light node	Transparent delegation with cost and balance; per-provider ledgers; deterministic `payment_required` on exhaustion; resume after credit	Trusted-operator only (Section 11)
Target frontier deployment	GLM-5.2-class	Analytical sizing only	Not run

The abstract's headline, the "Paris." completion, should be read as expected, token-identical output under a partial store, demonstrating the fetch-verify-persist loop and engine correctness, not as a factual-accuracy evaluation.

Measurement agenda (not yet done). Tokens per second as a function of miss rate and NIC tier; repair time after killing a sole holder; the committee-saturation join sequence at scale; and end-to-end throughput once hardware-tailored kernels land.

---

11. Limitations

Consolidated so a reader sees the full picture in one place.

Compute and coverage.

- Kernels are scalar (no SIMD or GPU), about 0.3 tokens/sec on a 15.7B model on one core.
- Single-sequence only; no continuous batching.
- GLM 5.2 itself has not been run; the deepseek2 path is the architectural proxy.
- No wide-area or contended-network measurements exist.

Trust (v1 is operator-trusted).

- Manifest choice is trust-on-first-use; a hostile bootstrap can pin a poisoned root.
- Holdings claims are unverified; committee completeness assumes honesty.
- No transport encryption or authentication (plaintext TCP), which is LAN-appropriate, not public.
- Availability attacks (service refusal, mesh poisoning, flooding) are not defended.

Metering (trusted-operator accounting demo).

- Client ids are self-asserted strings, so rotating an id resets the free quota.
- Ledgers are per-provider, so round-robin across N full nodes yields N times the free quota.
- `credit` is admin-gated but its payment proof is unverified in v1.
- No signed receipts, so neither party can prove the other's ledger wrong.
- Prerequisites for real compensation: cryptographic client identity, payment-proof verification, signed usage receipts.

Operational.

- The distributed engine is exercised via `loom gguf run`, not yet the node's RPC path.
- Persisted organic replicas have no disk cap or eviction yet, so a hot node's disk grows.
- ENR advertising and hardfork coordination are designed but unimplemented.

---

12. Roadmap

Milestones with exit criteria; full tracking in ROADMAP.

1. **Hardware-tailored kernels** (issues #10 to #14). Exit: a measured tokens/sec improvement over scalar on DeepSeek-V2-Lite on a single node, via a backend interface with a scalar differential oracle; CPU SIMD first, then GPU.
2. **Serve the distributed engine through node RPC.** Done: `loom node` serves the distributed GGUF (deepseek2) engine over its RPC and OpenAI surfaces, answering inference requests via the token-loop peer fetch, not only `loom gguf run`. A partial-store node's output is byte-identical to a full-store node's for the same prompt.
3. **Network-tier evaluation.** Exit: a published table of tokens/sec vs miss rate across 1 and 10 GbE, and repair time after a sole-holder failure.
4. **Hardfork coordination.** Exit: a majority of nodes adopting a new manifest version, with the existing version guard enforcing isolation during transition.
5. **v2 trust layer.** Exit: untrusted-peer serving with pollution-resistant propagation and sampled compute verification.

---

13. Conclusion

Loom reframes frontier-MoE inference as a distributed-storage problem: store the expert corpus once across a swarm, verify it cryptographically, and page it into node-local computation on demand. The consequences (soft failure, a compounding cache, and self-healing committee membership) follow from moving immutable weights rather than mutable activations. The implementation demonstrates the core loop end-to-end on real weights under a partial store, and is explicit about the boundary: today's result is correctness and integration on localhost with scalar kernels, and the pitch is capacity, not speed. Pooling ordinary machines lets a frontier MoE model fit where it otherwise could not. Making it fast is a separate problem, bound by network and kernels, and the roadmap treats it as such.

References

1. DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* 2024. https://arxiv.org/abs/2405.04434 (Multi-head Latent Attention).
2. DeepSeek-AI. *DeepSeek-V3 Technical Report.* 2024. https://arxiv.org/abs/2412.19437
3. Moonshot AI. *Kimi K2.* 2025. https://github.com/MoonshotAI/Kimi-K2

4. GLM Team, Zhipu AI. *ChatGLM / GLM-4 family.* 2024. https://arxiv.org/abs/2406.12793 (GLM 5.2 is the design target; the GLM-4 report is the cited real family.)
5. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models.* 2022. https://arxiv.org/abs/2209.01188
6. exo-explore. *exo: run your own AI cluster at home.* https://github.com/exo-explore/exo
7. Gerganov et al. *llama.cpp / GGML / GGUF.* https://github.com/ggml-org/llama.cpp
8. Song et al. *PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU.* 2023. https://arxiv.org/abs/2312.12456
9. Sheng et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU.* 2023. https://arxiv.org/abs/2303.06865
10. Merkle, R. *A Digital Signature Based on a Conventional Encryption Function.* CRYPTO 1987 (Merkle trees and content addressing).
11. Peng et al. *YaRN: Efficient Context Window Extension of Large Language Models.* 2023. https://arxiv.org/abs/2309.00071
12. Ethereum. *EIP-778: Ethereum Node Records (ENR).* https://eips.ethereum.org/EIPS/eip-778 ; libp2p gossipsub. https://github.com/libp2p/specs/tree/master/pubsub/gossipsub
13. Google. *Snappy compression.* https://github.com/google/snappy (Zig binding: https://github.com/blockblaz/zig-snappy)
14. Krohn, Freedman, Mazieres. *On-the-fly Verification of Rateless Erasure Codes for Efficient Content Distribution.* IEEE S&P 2004 (homomorphic hashing).

Appendix A: Decision log

For contributors. This append-only log records design decisions with dates and rationale. It is maintained alongside the code and is not part of the paper's narrative. Superseded decisions are struck through, not deleted. Where this log, the spec, and the roadmap differ, the spec governs the p2p protocol and the roadmap governs status.

Date	Decision	Rationale
2026-07-11	Distribution unit is the expert, not the layer; compute node-local; never distribute KV	MoE sparsity plus MLA's tiny KV; avoids activations in the serial network path
2026-07-11	Content-addressed shards plus a Merkle-rooted manifest	Free integrity and poisoning check at every hop; root doubles as version id
2026-07-11	No coding (EC or RLNC) in the token loop, ever	Decode-before-use adds latency where it cannot be hidden; coding stays in propagation and durability
2026-07-12	Target toolchain: Zig 0.16.0, matching the sibling zeam project	Shared toolchain plus reusable deps (ENR, SSZ, snappy)
2026-07-12	v1 direction: GGUF interchange format; ENR, gossip, and request-response p2p; majority hardforks; boot-time peer sync	Requirements of record; supersedes an earlier Hyperswarm/Hyperbee sketch
2026-07-12	Weight advertising: both ENR (compact summary within the 300 B limit, a bitmap digest plus seq) and a global gossip topic (full bitmap)	ENR for discovery-time selection; gossip for live freshness
2026-07-13	Churn repair is maximally eager; an always-on loop retries all known peers and dead peers are retried, not forgotten	Chosen over lazy repair-on-miss
2026-07-13	Gossip is epidemic announce-and-exchange over the peer table; transport swappable for gossipsub later	Table semantics are transport-independent

Date	Decision	Rationale
2026-07-19	deepseek2 rope is NORM (adjacent-pair), not NEOX	Empirical: DeepSeek's (d/2,2) transpose before rotate_half nets to adjacent-pair on the stored layout; NEOX degenerates
2026-07-19	Response payloads carry no digest; the requester verifies against its own manifest	The manifest is the only trust root
2026-07-19	Shard equals one expert block plus mandatory 16 MB resident chunks; about 19 MB per expert gives 19,200 shards for GLM 5.2	Shard unit must equal the matmul's consumption unit; metadata stays trivial
2026-07-19	Bootnode assigns committees least-covered-first toward R = 3 target, R = 2 floor; saturated committees spawn new ones	Coverage by construction; bootnode out of the inference path
2026-07-19	Fetched shards are persisted, not evicted, so fetch-on-demand is organic heat replication	Disk is the cheap resource; hot experts gain holders by being used
2026-07-20	Wire messages v1: binary frames, adaptive snappy; heartbeat carries seq, digest, and load but not the bitmap	Quantized payloads are incompressible; heartbeats stay small; staleness detected cheaply
2026-07-20	Committee membership is gossip-derived, since announces carry committee id	Fixes static-at-join membership; survives bootnode death
2026-07-20	Two node classes: light nodes (no weights or engine; delegate) and full nodes (shards plus inference)	Local inference for low-memory devices without weakening supply
2026-07-20	Compensation: per-client metering ledger on each full node; `payment_required` enforcement; `credit` op as the settlement seam	Accounting precedes payment rails; ledgers are per-provider
2026-07-20	Redundancy R = 3 target, R = 2 floor (default 2); completeness means R >= 1	Completeness and redundancy are distinct thresholds
2026-07-20	Placement is least-covered-first committee assignment; random `--hold-fraction` is the no-bootnode fallback	One live placement policy
2026-07-20	Binary frames are normative; the line protocol is legacy and debug during migration	Prevent forked implementations
2026-07-20	Hardware-tailored compute backends behind one `Backend` seam (scalar oracle; CPU SIMD then GPU); planned as issues #10 to #14	Per-node throughput gates the distribution story
2026-07-20	Code-audit hardening: version root binds layout; hot-path and serve re-verify; store-open re-audits held shards; resident completeness gate; publish-after-verify cache; credit admin-gated; light node forces client id; token clamp; account cap; snappy decode cap; bounded peer table; monotonic holdings seq; connection caps; death triggers wanted re-replication	Close the gap between "verified before use" and the code; ship the availability bounds the spec named
2026-07-20	Documents restructured to standard whitepaper format (front matter, TOC, glossary, related work, references, claim-to-evidence table, limitations); terminology split (expert shard vs	Editorial reviews: read as a whitepaper, scope claims precisely, cite external work

Date	Decision	Rationale
	resident chunk); decision log demoted to a contributor appendix	
2026-07-20	Add an OpenAI-compatible HTTP API as the north-facing client surface (`--openai-port`); client identity moves to `Authorization: Bearer`; MCP is not the serving path (a node is more naturally an MCP client)	Ecosystem clients work with no adapter; bearer identity is out-of-band and not prompt-forgeable, unlike the native RPC `client` field. Shipped as a skeleton (`src/node/openai.zig`): transport, routing, shapes, identity; generation/streaming TODO
2026-07-21	A `Generator` abstraction (`src/node/generator.zig`) unifies the loom-format and distributed GGUF (deepseek2) engines; both RPC and OpenAI serve through it; the node auto-serves the GGUF engine when an expert-sharded store is attached and its resident bundle is complete	Serves the distributed engine (token-loop peer fetch) through the node, not only `loom gguf run` (roadmap item 2). Store mutation from serving-fetch and eager repair serialize on one engine mutex
2026-07-21	Light nodes delegate the OpenAI surface too (`src/node/light_openai.zig`), a metered reverse proxy to full-node OpenAI endpoints; the light node forces its client id via `Authorization: Bearer` (caller bearer dropped)	OpenAI clients can point at a low-memory light node; identity is forced so a light node cannot be an open proxy, parallel to the native-RPC rule
2026-07-22	SSE streaming for `stream:true` (`src/node/openai.zig`): OpenAI `text/event-stream`, one chunk per token then `[DONE]`, over both engines. Backed by an optional per-token callback on `engine.generate` and a `generator.TokenSink`	Chat clients that default to streaming work; client disconnect aborts generation, metering still charged
2026-07-22	Per-model chat templates (`src/gguf/chat_template.zig`): detect the format from GGUF `tokenizer.chat_template` (or arch), render `messages[]` per format (deepseek/chatml/llama2/llama3/gemma/mistral/generic), `--chat-format` override. Format detection plus built-in renderers, not a Jinja engine	Real chat models need their own prompt format. DeepSeek-V2 (the validated target) is faithful
2026-07-22	Special-token-aware tokenization (`src/gguf/special.zig`): both BPE and SPM splice control (type 3) / user-defined (type 4) tokens to atomic ids, longest-match with a first-byte filter, splitting the input before normal encoding	Chat markers (chatml/llama3/gemma) and control tokens tokenize exactly instead of being split. No change when no special appears (no regression). Caveats: parse_special always on (injection); SPM dummy-prefix on the first segment only
2026-07-24	Parse-special injection hardening: `parse_special` threaded through the tokenizer chain; off for raw prompts and text-marker chat formats (deepseek/llama2/mistral), on only for special-marker scaffolds (chatml/llama3/gemma)	Untrusted input can no longer inject a control token on the raw-prompt or RPC path, nor via text-marker chat content (the validated DeepSeek-V2 target is fully safe). Segment-encoding content for special-marker formats is the remaining follow-up

Date	Decision	Rationale
2026-07-27	Codebook quantizations implemented (`src/gguf/iq.zig`): IQ1_S/M, IQ2_XXS/XS/S, IQ3_XXS/S, IQ4_NL/XS and MXFP4, with grid and sign tables transcribed mechanically from llama.cpp (`src/gguf/iq_tables.zig`)	Most modern GGUF repositories publish mostly IQ variants, so the affine-only kernel set excluded the majority of available checkpoints. MXFP4 additionally covers microscaling checkpoints. Verified bit-for-bit against llama.cpp's own `dequantize_row_*` on stored vectors (`src/gguf/iq_vectors.zig`), because a wrong codebook entry corrupts weights silently instead of failing
2026-07-27	MoE routing extracted to `src/gguf/moe.zig` and shared by both engines, including the expert-to-shard binding used for distributed fetch	The routing was never DeepSeek-specific: upstream funnels every MoE architecture through one routing function differing in four knobs (gating function, selection bias, gate renormalization, constant scale). Sharing it means a new architecture needs an attention variant, not a new engine
2026-07-27	The llama engine generalized into one GQA engine covering `llama` (Mixtral), `qwen2moe`, `qwen3moe` and `glm4moe`; optional features detected from the file's tensors, with only the RoPE style pinned per architecture	Distributed serving previously required `deepseek2`, even though sharding, sync and repair were already architecture-agnostic — a Mixtral store distributed correctly but could not be served. Feature detection follows the checkpoint rather than a table of beliefs about each architecture, which is also what makes the qwen3 explicit `head_dim` and the glm4 NextN skip fall out naturally. Verified end to end: for all five architectures a node holding roughly 30% of shards serves with hit rate below 1 and produces output token-identical to a full-copy origin
2026-07-28	A node serves a GGUF locally when it is not expert-sharded, instead of falling back to the synthetic checkpoint	A dense model has no routed experts, so it shards into fixed ranges and cannot be served *distributed* — but it is a complete model file, and answering from an unrelated synthetic checkpoint reads as a broken model rather than as an unsupported topology
2026-07-28	A chat UI is compiled into the binary and served on its own listener (`--ui-port`, default 8555), sharing the HTTP implementation and generator with the OpenAI API; plus a periodic console status line (`--status-secs`)	Nothing in the system reported its own state: exercising a node meant hand-writing HTTP, and membership and holdings move with no request arriving, so an event-only log made a churning node look idle. Serving the page same-origin with the API removes any CORS or host configuration. The UI separates decode speed from time-to-first-token, because on a partial node the first token also pays for the prefill's expert fetches
2026-07-28	`/health` reports whether the served weights are one of loom's random-weight fixtures, and the UI says so	A fixture answers with meaningless text by construction; unlabelled, that is indistinguishable from a broken model and was the first thing a new user saw

Date	Decision	Rationale
2026-07-28	Q4_1 and Q5_1 implemented, completing every non-ternary GGML type	Three Q5_1 tensors out of 377 blocked an entire 8.9 GB checkpoint. llama.cpp quant *mixes* fold a handful of legacy types into otherwise K-quant files, so one missing format costs a whole model
2026-07-28	SIMD kernels via `@Vector` plus row-parallel matvec over a worker pool (issue #11)	Inference ran at roughly 1% of the machine's memory bandwidth on one of ten cores: the limit was scalar single-threaded code, not the absence of a GPU. Measured 23.6x on a 10-core M5 (1.1 -> 26.0 tok/s, TinyLlama 1.1B Q4_K_M). Row-splitting is exact rather than approximate, since each output row is an independent dot product with no reassociation, so results stay bit-identical to the serial path and the tests assert it. The pool is scoped to a generation because Zig 0.16 moved condition variables under `Io`, which the kernels deliberately do not depend on, so parked workers would otherwise spin
2026-07-28	int8-quantized activations for the Q4_K, Q6_K and Q8_0 kernels	Profiling showed 76 to 92 percent of a quantized matvec was the dequantize step, not the dot: each block was expanded into a 256-float scratch buffer and pushed through L1 only to be read back. Quantizing the activation once per matvec and dotting against packed weights as integers removes the buffer and uses lanes four times as wide. Measured 1.1 -> 51.9 tok/s overall (47x from the original scalar path). Unlike the earlier steps this is an approximation, about 0.4 percent per activation element, so results are no longer bit-identical to the dequantize path and the tests bound the error against the mass of the summed terms rather than against the heavily cancelled result
2026-07-28	Batched prefill (`stepBatch`, `ggml.matmul`) and vectorized attention inner loops	Decoding unpacks every weight once per token, but a prompt is known up front, so one unpacked weight can serve several tokens. Measured 1.4x on a 145-token prefill; the kernel microbenchmark shows 2.4x in isolation, and the gap is attention, which is quadratic in prompt length and bound by the KV cache rather than by weight reads, so there is nothing to amortize. Vectorizing the attention dot and value-accumulation loops was worth a further 11% on decode. Batch capped at 8: past that, register pressure costs more than the sharing returns

Appendix B: Source layout

```
spec/          protocol specification (SPEC.md)
docs/          roadmap and planning
whitepaper/    this document
src/main.zig   CLI entry
src/core/      hashing/Merkle, tensor math, int4 quant, stats, iobench
src/engine/    loom-format MoE engine (MLA, router, expert cache, checkpoints)
src/gguf/      GGUF parser, GGML + codebook kernels, MLA and GQA engines, shared MoE routing, BPE
src/p2p/       distribution: wire frames, gossip, committees, sync, token-loop fetch
src/node/      daemon: node orchestration, RPC, model resolver, light node, metering
```

Appendix C: Provenance

Loom's design draws on the colibri expert-streaming technique; llama.cpp and GGML for the GGUF format, quantization layouts, and reference kernel semantics [7]; the DeepSeek V2/V3 model architecture (MLA, sparse routing) [1][2]; Ethereum networking patterns (bootnodes, ENR, gossip, committees) [12]; and content-addressed storage (Merkle verification, version-pinned manifests) [10]. The implementation is original Zig with a single dependency (blockblaz/zig-snappy [13]).